

Below is your full **SSO using Token between two Oracle APEX applications** converted into a clean, professional **documentation format**, ready to share or include in a project document.

Oracle APEX Token-Based SSO Implementation Guide

Overview

This document describes how to implement a **Token-Based Single Sign-On (SSO)** between two Oracle APEX applications (Master and Child) using database tokens.

The Master application generates a secure token and redirects the user to the Child application. The Child application validates the token and allows login without re-entering credentials.

Architecture

Flow:

1. Master App generates a token
2. Token stored in DB
3. Redirect to Child App with token
4. Child App validates token
5. User authenticated

Prerequisites

- * Oracle APEX installed
- * Access to both APEX applications
- * Common database or database link between schemas
- * Ability to create tables and packages.

Step 1: Create Token Table

Create this table in:

- * Both schemas, ****if Master and Child use different schemas****
- * Or only once in master schema, ****if both apps use the same schema****

```
CREATE TABLE SSO_TOKENS (  
  TOKEN_ID    VARCHAR2(200),  
  USERNAME    VARCHAR2(100),  
  CREATED_ON  DATE DEFAULT SYSDATE,  
  EXPIRES_ON  DATE,  
  IS_USED     CHAR(1) DEFAULT 'N',  
  CONSTRAINT PK_SSO_TOKENS PRIMARY KEY (TOKEN_ID)  
);
```

Purpose

This table stores temporary login tokens for validating SSO requests.

Step 2: Create SSO Package Specification

```
CREATE OR REPLACE PACKAGE PKG_APEX_SSO AS
FUNCTION CREATE_TOKEN (p_token VARCHAR2, p_username VARCHAR2) RETURN VARCHAR2;
FUNCTION VALIDATE_TOKEN (p_token VARCHAR2) RETURN VARCHAR2;
END PKG_APEX_SSO;
/
```

Step 3: Create SSO Package Body

```
CREATE OR REPLACE PACKAGE BODY PKG_APEX_SSO
AS
    FUNCTION CREATE_TOKEN (p_token VARCHAR2, p_username VARCHAR2)
        RETURN VARCHAR2
    IS
    BEGIN
        INSERT INTO SSO_TOKENS (TOKEN_ID,
                                USERNAME,
                                CREATED_ON,
                                EXPIRES_ON)
            VALUES (p_token,
                    p_username,
                    SYSDATE,
                    SYSDATE + (5 / 1440));           -- valid for 5 minutes

        COMMIT;
        RETURN p_token;
    END;

    FUNCTION VALIDATE_TOKEN (p_token VARCHAR2)
        RETURN VARCHAR2
    IS
        v_user    VARCHAR2 (100);
    BEGIN
        SELECT USERNAME
            INTO v_user
            FROM SSO_TOKENS
           WHERE TOKEN_ID = p_token AND IS_USED = 'N' AND EXPIRES_ON >= SYSDATE;

        UPDATE SSO_TOKENS
           SET IS_USED = 'Y'
          WHERE TOKEN_ID = p_token;

        COMMIT;
        RETURN v_user;
    EXCEPTION
        WHEN NO_DATA_FOUND
        THEN
            RETURN NULL;
    END;
END PKG_APEX_SSO;
---
```

Notes

- * Token expires in 5 minutes
- * Token can be used only once
- * Expired or reused tokens are rejected

Step 4: Build Redirect URL in Master App

Generate token and redirect user to Child App.

Example Redirect URL

```
:VC_APP_URL
|| '?P9999_TOKEN=' ||: P4_G_SSO_TOKEN
|| '&P9999_USERNAME=' ||: APP_USER
...
```

Where

- * `P4\_G\_SSO\_TOKEN` → Random token generated in Master App
- * `VC\_APP\_URL` → Child app login page URL
- * `:APP\_USER` → Logged in user

Step 5: Prepare Child App Login Page

Page Settings

- * Login Page (e.g. Page 9999)
- * **Page Access Protection** → Unrestricted

Create Page Items

Item Name	Type	Purpose
-----	-----	-----
P9999_TOKEN	Hidden	Receives SSO token
P9999_USERNAME	Hidden	Receives username

Add Before Header Process

Process Name: `INIT\_SSO\_TOKEN`

```

DECLARE
    v_token    VARCHAR2 (200);
    v_count    NUMBER;
BEGIN
    BEGIN
        SELECT COUNT (*)
            INTO v_count
            FROM SSO_TOKENS
            WHERE TOKEN_ID = :P9999_TOKEN;
    EXCEPTION
        WHEN OTHERS
        THEN
            v_count := 0;
    END;

    IF v_count = 0
    THEN
        v_token := PKG_APEX_SSO.CREATE_TOKEN ( :P9999_TOKEN, :P9999_USERNAME);
    END IF;
END;
---
```

Step 6: Create Custom Authentication Scheme

Settings

* **Authentication Type** → Custom

* **Authentication Function Name** → `CUSTOM\_AUTH`

Custom Authentication Function

```

FUNCTION custom_auth (
    p_username IN VARCHAR2,
    p_password IN VARCHAR2
)
RETURN BOOLEAN
AS
    v_user VARCHAR2(100);
BEGIN
    -- Token based authentication
    IF :P9999_TOKEN IS NOT NULL THEN
        v_user := PKG_APEX_SSO.VALIDATE_TOKEN(:P9999_TOKEN);

        IF v_user IS NOT NULL THEN
            RETURN TRUE;
        ELSE
            RETURN FALSE;
        END IF;
    END IF;

    RETURN FALSE;
END;
```

Security Best Practices

- * Token expiry enforced (5 minutes)
- * Token single-use only
- * Do not expose token in browser logs unnecessarily
- * Use HTTPS only
- * Clean expired tokens via scheduled job

Optional: Cleanup Job for Old Tokens

```
DELETE FROM SSO_TOKENS
WHERE EXPIRES_ON < SYSDATE;
COMMIT;
```

Schedule via DBMS_SCHEDULER.

Common Issues & Troubleshooting

Issue	Cause	Solution	
-----	-----	-----	
Token not working	Expired	Increase expiry time	
Invalid token	Already used	Generate fresh token	
User not logged in	Wrong redirect URL	Verify page items	
ORA- errors	Missing grants	Grant table/package access	

Summary

This token-based SSO approach allows seamless authentication between Oracle APEX applications without re-login, using:

- * Secure tokens
- * Custom authentication
- * Controlled expiry and usage